

플랜트 배관 3D 직교 라우팅 엔진 — 개발 계획서

Python 알고리즘 실험 → C++ 엔진화 (pybind11 연계) 작성일: 2026-05-27 버전 1.0

목차

- 프로젝트 개요
- 현재까지의 진행 현황
- 단계별 개발 계획
- 실험-엔진 연계 방식 비교
- C++ / Python 특성 비교
- 주요 리스크 및 대응
- 마일스톤 (예시)

1. 프로젝트 개요

1.1 목표

플랜트 환경의 BIM 데이터로부터 메인 장비 ↔ 종단 POC 사이의 3D 직교 자동 배관 경로를 산출하는 라우팅 엔진을 개발한다.

1.2 개발 전략 (2트랙)

알고리즘 실험과 프로덕션 엔진을 분리해 두 트랙으로 진행한다.

- Python 트랙** — 비용함수·제약·우선순위 규칙을 빠르게 반복 검증한다. 핵심 지표는 실행 속도가 아니라 반복 속도.
- C++ 트랙** — 확정된 알고리즘을 프로덕션 수준의 엔진으로 구현하고, pybind11 바인딩으로 Python 실험 환경과 영구 연결한다.

2. 현재까지의 진행 현황

프로토타입 수준에서 다음이 이미 구현·검증되었다.

구성요소	내용	상태
C++ 라우팅 프로토타입	직교 A* + turn penalty + 클리어런스 inflation + sparse 점유맵 + 다중 배관	검증 완료
방문맵 수집 / Export	A* 탐색 셀 수집, scene.txt(점유·방문·경로) 내보내기	검증 완료
C# 시각화 뷰어	HelixToolkit WPF, 레이어 토글, 복셀 머지 렌더링	코드 완성

이 자산들은 Python 실험 단계의 비교 기준(baseline)이자, C++ 엔진화 단계의 출발점이 된다.

3. 단계별 개발 계획

3.1 1단계 — Python 알고리즘 실험

목적: 성능 부담 없이 알고리즘과 파라미터를 빠르게 반복 검증한다. 이 단계의 핵심 지표는 실행 속도가 아니라 반복 속도다.

작업	내용	도구/라이브러리
점유맵 구현	NumPy 불리언 배열 또는 dict 기반 sparse 맵	NumPy
A* 탐색 포팅	C++ 직교 A* 로직을 heapq 기반으로 미러링	heapq, dataclasses
비용함수 실험	turn penalty, 클리어런스, 단 분리 가중치 튜닝	직접 구현
다중 배관 전략	라우팅 순서·우선순위 규칙 비교	직접 구현
핫스팟 가속	탐색 루프 등 병목만 선택적 가속	Numba
시각화	점유·방문·경로 즉석 3D 렌더	PyVista / Plotly

산출물: 검증된 알고리즘 파라미터 셋, 실험 노트, scene.txt 호환 입출력, 비교 기준 데이터.

3.2 2단계 — 인터페이스 동결

목적: 실험으로 확정된 알고리즘·제약·데이터 포맷을 명세로 고정하여 C++ 포팅의 설계 변경 리스크를 제거한다.

- 비용함수·제약·우선순위 규칙을 문서화하여 명세로 확정한다.
- 중간 데이터 포맷(scene.txt 등)을 정식 규격으로 동결한다.
- 대표 시나리오를 회귀 테스트 케이스로 고정한다(입력 → 기대 경로/지표).
- 성능 목표치(셀 규모별 목표 시간·메모리)를 수치로 정의한다.

산출물: 알고리즘 명세서, 데이터 포맷 규격서, 회귀 테스트 세트, 성능 목표 정의.

3.3 3단계 — C++ 엔진화 + pybind11

목적: 확정된 알고리즘을 프로덕션 수준의 C++ 엔진으로 구현하고, Python 바인딩으로 실험 환경과 연결한다.

영역	구현 방향	라이브러리
점유맵	희소 복셀 기반으로 8,000m 스케일 대응	OpenVDB
탐색 자료구조	decrease-key 지원 우선순위 큐	Boost.Heap
방문/closed 맵	고속 해시맵으로 교체	abseil flat_hash_map
충돌검사	메시-프리미티브 간섭 검사	FCL
다중 배관 조정	선점 문제 해소 (rip-up & reroute / CBS)	직접 구현
Python 바인딩	C++ 엔진을 Python에서 호출	pybind11

산출물: C++ 라우팅 엔진, Python 바인딩 모듈, 성능 벤치마크 결과, 회귀 테스트 통과 리포트.

4. 실험-엔진 연계 방식 비교

Python 실험 자산을 어떻게 처리할지에 대한 선택지와 권장안이다.

접근	적합한 경우	평가
Python 실험 → C++ 완전 재작성	실험과 프로덕션을 분리하고 실험 코드는 폐기	단순
Python 실험 → C++ 코어 + pybind11	실험 환경을 회귀 테스트·튜닝 도구로 영구 보존	권장
Python + Numba/Cython 유지	중간 규모까지만 필요하고 C++ 전환을 미룰 때	한시적

5. C++ / Python 특성 비교

항목	C++ (엔진)	Python (실험)
실행 성능	대규모 탐색에 압도적	순수 실행은 느림 (가속 필요)
개발/반복 속도	느림 (빌드·타입 부담)	매우 빠름
메모리 효율	직접 제어, 오버헤드 최소	객체 오버헤드 큼
비용함수 튜닝	재컴파일 필요	즉시 수정·재실행
공간/수치 생태계	OpenVDB·FCL·CGAL (강력·무거움)	NumPy·SciPy·Open3D (간편)
시각화	별도 뷰어 필요	즉석 3D 렌더 용이
병렬화	스레드·SIMD 직접 활용	GIL 제약 (우회 필요)
배포	단일 실행파일	인터프리터·패키지 환경 필요

6. 주요 리스크 및 대응

리스크	영향	대응
8,000m 스케일 성능 미달	실용성 저하	계층 corridor + OpenVDB 조기 검증
다중 배관 선점 충돌	경로 품질 저하	rip-up & reroute / CBS 도입
Python→C++ 결과 불일치	회귀 신뢰성 저하	동일 시나리오 회귀 테스트로 교차 검증
방문맵 시각화 부하	뷰어 성능 저하	포인트 클라우드 + SharpDX 전환

7. 마일스톤 (예시)

실제 일정은 자원에 맞춰 조정한다. 아래는 단계 간 선후 관계를 보여주는 예시다.

단계	주요 산출물	완료 기준
1단계	Python 실험 환경, 파라미터 셋	대표 시나리오에서 안정적 경로 생성
2단계	명세서, 포맷 규격, 회귀 세트	알고리즘·포맷 동결 합의
3단계	C++ 엔진, pybind11 모듈	성능 목표 달성 + 회귀 통과

본 계획은 진행 상황에 따라 갱신한다. **다음 즉시 실행 항목**은 1단계 Python 실험 환경 구축 (C++ A* 로직 미러링 + PyVista 시각화) 이다.